

selenium-bot Projesi — Haftalık Çalışma Raporu

IDS/ML Staj Projesi · LIMOS-CNRS · Mustafa Göçmen · Temmuz 2026

İçindekiler

- 1. Genel Bakış — Bu Hafta Ne Yapıldı
- 2. Pi Altyapısı — Kalıcılık Sorunları ve Çözümleri
- 3. RITA — Kurulum, İlk Test, Kapsam Düzeltmesi
- 4. MixedTrafficRunner — Yeni Bir Trafik Üretim Aracı
- 5. Sorun: Kapasite ile Örneklem Arasındaki Bağ (Little's Law)
- 6. Bot Kodundaki İki Yarış Durumu (Race Condition) Hatası
- 7. Dosya Düzeni — Proje Klasörlerinin Organizasyonu
- 8. PRODUCT_DETAIL Navigasyon Hatası — En Derin Kazi
- 9. İstatistiksel Doğrulama Sonuçları (KS Testi)
- 10. RITA Beacon Testleri — Kapsamlı Araştırma
- 11. Genel Metodolojik Dersler
- 12. Sonuç ve Sıradaki Adımlar

1. Genel Bakış — Bu Hafta Ne Yapıldı

Bu hafta üzerinde çalıştığın `selenium-bot` projesi, bir Raspberry Pi üzerinde barındırılan sahte bir e-ticaret sitesine (`techmarket.lab`) karşı, gerçek bir insan kullanıcıyı taklit eden bot trafiği üretiyor. Bu trafiğin istatistiksel olarak gerçek CICIDS2017 veri setindeki insan trafiğine ne kadar yakın olduğunu ölçüyorsun, ve klasik ağ güvenlik araçlarının (Zeek, RITA) bu trafiği tespit edip edemediğini test ediyorsun. Nihai hedef, bu trafiği eğitim verisi olarak kullanacak bir Autoencoder (sinir ağı) ile "normal görünen ama aslında otomatik" trafiği yakalamak.

Bu hafta beş ana başlıkta çalışıldı:

- Altyapı kalıcılığı:** Raspberry Pi'nin bazı ayarlarının (sudo izinleri, DNS ayarları) reboot sonrası kendiliğinden bozulması sorunu kalıcı olarak çözüldü.
- Yeni bir trafik üretim aracı** (`MixedTrafficRunner`) tasarlandı ve inşa edildi — gerçekçi, sürekli, karışık bot trafiği üretmek için.
- Üç gerçek kod hatası** bulundu ve düzeltildi — ikisi eşzamanlılık kaynaklı yarış durumları, biri de botun "gerçekçilik" iddiasını zayıflatan bir navigasyon mantığı hatası.
- İstatistiksel doğrulama** tekrarlandı — düzeltmelerin botun CICIDS'e olan istatistiksel yakınlığını gerçekten iyileştirdiği kanıtlandı.
- RITA ile kapsamlı bir tespit testi** yapıldı — sonuç, botun net bir şekilde tespit edilemediğini, ama kesin bir "hayır" da diyemediğini gösterdi.

Aşağıda her başlığı, **neden yaptığımızı, nasıl bulduğumuzu, ne öğrendiğimizi** anlatacak şekilde detaylandırıyorum.

2. Pi Altyapısı — Kalıcılık Sorunları ve Çözümleri

Projenin can damarı olan Raspberry Pi'de, geçmiş günlerden kalan iki tekrarlayan sorun vardı: bazı ayarlar reboot sonrası kendiliğinden kayboluyordu. Bu hafta ikisini de kalıcı olarak çözdük.

2.1 Sorun: `tcpdump` için parolasız sudo izni kayboluyordu

Sorun

Pi'de trafik yakalamak (`tcpdump` çalıştırmak) için normalde parola girmen gerekmiyordu, çünkü bir sudoers kuralı (`NOPASSWD`) tanımlıydı. Ama bu kural bazen kendiliğinden kayboluyordu — nedeni tam olarak tespit edilemedi (muhtemelen bir sistem servisi ya da reboot sürecinde bir şey bu dosyayı sıfırlıyordu).

Çözüm

Nedeni bulamasak da, **sonucu garanti eden** bir çözüm kurduk: `restore-tcpdump-sudoers.service` adında bir systemd servisi yazıldı. Bu servis, Pi her açıldığında otomatik çalışıp "bu kural yerinde mi?" diye kontrol ediyor, yoksa yeniden yazıyor. Yani artık kuralın neden kaybolduğunu bilmesek de, her boot'ta kendini onarıyor.

2.2 Sorun: `/etc/hosts` 'taki bir satır reboot'ta geri geliyordu

Sorun

Daha önceki bir hatanın (geçmiş günlerde bulunan bir "loopback trafiği" hatası) kalıntısı olarak, Pi'nin `/etc/hosts` dosyasında `techmarket.lab` 'ı kendi kendine işaret eden bir satır vardı. Bu satırın orada OLMAMASI gerekiyordu (yoksa Pi kendi trafiğini kendine gönderip Zeek'in göremediği bir "loopback" hatasına düşüyordu). Ama Pi'nin işletim sistemindeki `cloud-init` adlı bir bileşen, bu satırı her reboot'ta otomatik olarak geri ekliyordu.

Çözüm

`cloud-init` 'in bu davranışını (`manage_etc_hosts`) tamamen kapattık — `/etc/cloud/cloud.cfg.d/99-disable-etc-hosts.cfg` dosyasına `manage_etc_hosts: false` eklendi. Reboot testiyle doğrulandı, satır artık geri gelmiyor.

2.3 Ekstra kontrol: Zaman senkronizasyonu (NTP)

Pi'nin saatinin doğru olup olmadığını da kontrol ettik, çünkü ileride log'ları zaman damgasına göre karşılaştıracamız. `timedatectl` komutu "senkron değil" diye yanlış bir uyarı veriyordu — ama gerçek nedeni araştırdığımızda, bu Pi'de `ntpsec` adlı farklı bir zaman senkron servisinin çalıştığını, ve `timedatectl` 'in bu servisi tanımadığı için "senkron değil" dediğini bulduk. `ntpq -p` komutuyla gerçek durumu kontrol ettik: saat gerçekten doğru senkronizeydi (offset sadece 0.14 milisaniye) — sadece görüntüleme aracı yanlış bilgi veriyordu.

Buradan öğrenilen ders

Bir aracın (`timedatectl`) verdiği bilgiye körü körüne güvenmemek gerekir — o araç sadece bildiği servisleri anlayabilir. Gerçek durumu, konuya özel doğru araçla (`ntpq`) kontrol etmek gerekti.

3. RITA — Kurulum, İlk Test, Kapsam Düzeltmesi

RITA nedir? RITA (Real Intelligence Threat Analytics), ağ trafiğini analiz edip "beacon" (düzenli aralıklarla dışarıya sinyal gönderen, kötü amaçlı yazılımların tipik davranışı) tespiti yapan açık kaynak bir araç. Projenin amacı, klasik araçların (Zeek, RITA gibi) botun trafiğini "normal" sanıp sanmadığını test etmektir.

3.1 İlk engel — RITA hiçbir şey görmüyordu

Sorun

RITA'yı çalıştırdığımızda hiçbir sonuç gelmiyordu — sanki hiç trafik yokmuş gibi. Sebebini araştırdığımızda, RITA'nın varsayılan ayarının, "iç ağdan iç ağa" (internal-to-internal) giden trafiği otomatik olarak analiz dışı bıraktığını bulduk. Bizim lab ağımız (192.168.10.0/24) tamamen "iç ağ" sayıldığı için, bot ile sunucu arasındaki TÜM trafik bu filtreye yakalanıp göz ardı ediliyordu.

Çözüm

RITA'nın config dosyasındaki `always_included_subnets` ayarına `192.168.10.0/24` 'ü ekleyerek, bu filtreyi bizim lab ağımız için devre dışı bıraktık. Bu, RITA'nın normal/production kullanımında mantıklı bir varsayılan davranış, sadece bizim izole test ortamımızda gerekli bir istisna.

3.2 İlk gerçek ölçüm

Bu düzeltmeden sonra ilk anlamlı ölçümü yapabildik: botun trafiğine RITA'nın verdiği "**Beacon Score**" (**beacon skoru**) **0.275** çıktı — RITA'nın eşiği 0.50 olduğu için bu, "tespit edilmedi" (Severity: None) demektir. İlk bakışta iyi bir haber gibi görünse de, bu hafta ilerleyen bölümlerde (Bölüm 10) bu sonucun tek başına yeterli bir kanıt olmadığını, çok daha dikkatli test etmemiz gerektiğini keşfettik.

4. MixedTrafficRunner — Yeni Bir Trafik Üretim Aracı

Projede zaten iki çalıştırma aracı vardı:

- `App.java` — tek bir bot, tek bir davranış profiliyle (gezinen/arayan/form dolduran) çalışır.
- `ParallelRunner.java` — üç bot'u (biri taneden) aynı anda, bir kerelik çalıştırır.

Bunların hiçbiri, gerçek bir web sitesinin **sürekli, karışık, üst üste binen** trafiğini temsil etmiyordu. Bu yüzden yeni bir araç tasarlandı: `MixedTrafficRunner`.

4.1 Tasarım — nasıl çalışıyor

Bu araç, belirlenen bir süre boyunca (örneğin 18 dakika), rastgele aralıklarla yeni "kullanıcı" (bot oturumu) başlatıyor. Her yeni kullanıcının hangi davranış profiline (gezinen/arayan/form dolduran) sahip olacağı, gerçekçi bir ağırlıkla rastgele seçiliyor: **%60 gezinen, %30 arayan, %10 form dolduran** — gerçek bir e-ticaret sitesindeki tipik kullanıcı huninin yansıması.

Aynı anda en fazla **3 bot** çalışabiliyor (Mac'in kaynaklarını zorlamamak için) — bu bir `Semaphore` (izin sayacı) ile kontrol ediliyor.

5. Sorun: Kapasite ile Örnekleme Arasındaki Bağ (Little's Law)

Bu, haftanın en derin matematiksel/mühendislik sorunu. Anlaşılması için önce bir kavramı açıklamam gerekiyor.

5.1 Little's Law nedir — basit bir benzetmeyle

Bir kafede 3 kasiyer olduğunu düşün. Eğer müşteriler kasiyerlerin işleyebileceğinden **daha hızlı** gelirse, kuyruk sürekli büyür ve asla bitmez. Bu ilişkiyi matematiksel olarak ifade eden formüle **Little's Law** denir: $L = \lambda \times W$ (kuyruktaki ortalama kişi sayısı = varış hızı $\times$ her kişinin harcadığı ortalama süre). Eğer bu sayı, mevcut kasiyer sayısını (kapasiteyi) aşarsa, sistem asla dengeye gelmez.

5.2 Bizde ne oldu

Sorun

MixedTrafficRunner 'ı 10 dakika çalıştırmasını istediğimizde, gerçekte **20 dakika** sürdü ve bazı bot oturumları "sleep interrupted" hatasıyla yarıda kesildi. Sebebini Little's Law ile hesapladık: botun oturumları (özellikle "gezinen" profili, ortalama ~92 saniye) ile varış hızı (ortalama her 9 saniyede bir yeni bot) çarpıldığında, sistemde ortalama **10.2 bot aynı anda** olması gerekiyordu — ama sadece 3 kapasite vardı! Bu oran ($\rho = 3.4$), kuyruğun sınırsız büyüdüğü anlamına geliyordu.

Çözüm

İki şey yaptık: (1) Varış hızını, botların gerçek ortalama süresine göre yeniden hesapladık — yeni varış aralığı ortalama 45 saniyeye çekildi, bu da ρ 'yi 0.68'e (güvenli bölge) düşürdü. (2) Zorla kesmeyi (`shutdownNow()`) tamamen kaldırdık — artık hiçbir oturum yarıda kesilmiyor, kuyruk doğal olarak boşalıyor.

Buradan öğrenilen ders

Bir sistemin "N dakika sürmesi gerekiyor" beklentisi, sadece süre parametresine değil, o sistemin **iç dinamiklerine** (kaç iş aynı anda işlenebiliyor, her iş ne kadar sürüyor) bağlıdır. Bunu gözle tahmin etmek yerine, gerçek bir formülle (Little's Law) hesaplamak, "biraz uzun sürdü" gibi belirsiz bir gözlemi kesin bir sayıya ($\rho=3.4$) çevirdi ve doğru çözümü net gösterdi.

6. Bot Kodundaki İki Yarış Durumu (Race Condition) Hatası

Yarış durumu (race condition) nedir? Bilgisayar programlarında, bir işlemin bir başka işlemin bitmesini beklemeden başlamasından kaynaklanan hatalardır. Genelde nadir, tahmin edilemez şekilde ortaya çıkarlar — tam da bu yüzden bulmak zordur.

6.1 Sepete-ekleme butonunun "alert" penceresini kaçırmaması

Sorun

Bot bir ürüne "sepete ekle" butonuna tıkladığında, sitede bir JavaScript uyarı penceresi (`alert()`) açılıyor ve bot bunu kabul ediyor. Kod, butona tıkladıktan **hemen sonra, hiç beklemeden** bu pencereyi kabul etmeye çalışıyordu. Tek bir bot çalışırken bu sorun çıkmıyordu, ama `MixedTrafficRunner` ile 3 bot aynı anda CPU'yu paylaşıncaya, pencere açılması gecikip bot "pencere yok" hatası veriyordu.

Çözüm

Kodun tıklamadan sonra pencereyi **en fazla 10 saniye bekleyerek** aramasını sağladık (`WebDriverWait + ExpectedConditions.alertIsPresent()`). Bilerek hata mesajını sessizce yutmadık (try/catch ile gizlemedik) — çünkü sessizce yutmak, "sepete eklendi" diye işaretlenen bir oturumda gerçekte hiç eklenmemiş olma riski taşırdı; bu da ileride eğiteceğin modelin yanlış öğrenmesine yol açabilirdi.

6.2 Tıklamanın "görünür ama tıklanamaz" bir elemente gitmesi

Sorun

Botun her tıklama işlemi, önce elementin "görünür" olmasını bekliyordu (`visibilityOfElementLocated`). Ama "görünür olmak" ile "tıklanabilir olmak" farklı şeyler — bir buton görünür olabilir ama üstünde başka bir şey olabilir, ya da henüz aktif (enabled) olmayabilir. Yoğun eşzamanlı yükte, bir tıklama böyle bir elemente "gitti" ama hiçbir etki yaratmadı.

Çözüm

Bekleme koşulunu `elementToBeClickable` 'a yükselttik — bu, görünürlüğün yanında "aktif ve tıklanabilir" olmayı da garanti ediyor. Bu değişiklik, sadece bir yeri değil, botun **her** tıklamasını güçlendirdi.

Buradan öğrenilen ders

Bu iki hata da, tek bir bot çalışırken hiç görünmedi — sadece **gerçekçi, yoğun** koşullar altında (birden fazla bot aynı anda) ortaya çıktı. Bu, yazılım testinde önemli bir prensip: bir kodun "çalıştığını" düşünmek için onu sadece basit/izole koşullarda değil, gerçek kullanım yoğunluğunda da test etmek gerekir.

7. Dosya Düzeni — Proje Klasörlerinin Organizasyonu

Hafta ortasında, masaüstünde biriken dosyaların (capture'lar, analiz scriptleri, raporlar, deney log'ları) hangisinin ne işe yaradığı belirsizleşmişti. Her şeyi tek tek inceleyip kategorize ettik:

Klasör/Dosya Türü	Ne İşe Yarıyor
selenium-bot/	Botun kaynak kodu (git deposu, GitHub'a bağlı)
IDS-Analysis/captures/	Ham ağ trafiği kayıtları (.pcap) ve bunların sayısal özetleri (.csv)
IDS-Analysis/ANALYS-BENIGN/	CICIDS2017 istatistiksel analizi ve karşılaştırma scriptleri
IDS-Analysis/experiment-logs/	MixedTrafficRunner deney koşularının çıktı log'ları
context.md	Projenin tüm geçmişinin kronolojik özeti (bu belgenin "hafızası")

Ayrıca, güvenle silinebilecek gerçek çöp dosyaları da (macOS'un otomatik .DS_Store dosyaları, eski test kalıntıları, kirlenmiş eşzamanlı koşu log'ları) tespit edip temizledik.

Buradan öğrenilen ders

Bir projede "gereksiz görünen" dosyaları silmeden önce, her birinin gerçekten neyi temsil ettiğini anlamak önemli — bazı dosyalar (örneğin istatistiksel doğrulama raporları) küçük görünse de, o çalışmanın **tek kanıtı** olabilir.

8. PRODUCT_DETAIL Navigasyon Hatası — En Derin Kazı

Bu, haftanın en öğretici bölümü — çünkü sadece bir hata bulup düzeltmedik, aynı zamanda **"nasıl düzeltmeliyiz"** sorusunu da derinlemesine tartıştık.

8.1 Hatanın bulunuşu

İstatistiksel doğrulama testinde (`action_level_comparison.py`), "gezinen" bot profili sürekli olarak CICIDS'ten **istatistiksel olarak farklı** çıkıyordu (KS testi p-değeri ≈ 0). Kodu incelediğimizde şunu bulduk: bot bir ürüne tıklamadan önce, **hangi sayfada olduğuna bakmadan** her zaman önce ürün listesi sayfasını yeniden açıyordu. Ama Markov geçiş modelinde, ürün detayına giden en yaygın yol (%55-80 olasılık) zaten **ürün listesi sayfasındayken** gerçekleşiyordu — yani bot, zaten açık olan bir sayfayı gereksizce yeniden yüklüyor, ve bunu **hiç beklemeden** yapıyordu. Bu, hem gerçekçi olmayan bir davranış (gerçek bir insan, zaten gördüğü bir listeyi yeniden yüklemekten ürüne tıklar) hem de istatistiksel bir bozulma kaynağıydı.

8.2 İlk düzeltme, ve neden yeterli olmadığı

İlk düzeltmemiz basitti: "eğer zaten ürün listesindeyse, yeniden açma." Bu, sorunun büyük kısmını (dominant geçiş yolunu) çözdü ama küçük bir kısmı (kullanıcı listeyi hiç görmemişse giden azınlık yol) hâlâ aynı sorunu taşıyordu.

8.3 "Yol 1 mi Yol 2 mi" tartışması — bu haftanın en değerli düşünce süreci

Kalan sorunu çözmek için iki yol tartışıldı, ve bu tartışma gerçekten önemli bir prensibe dokunuyor:

	Yol 1 — Botun kodunu düzelt	Yol 2 — Sadece ölçüm/analiz script'ini düzelt
Ne yapılır	Eksik olan bekleme süresini bot koduna ekle	Bot koduna dokunma, sadece karşılaştırma script'i bu deseni "ayrı bir şey" say
Gerçek trafik değişir mi	Evet	Hayır
Risk	Doğrulanmamış bir varsayım eklenirse, bu kendi başına bir "bot imzası" olabilir	Gerçek bir kusur (sıfır-bekleme deseni) veride kalıcı olarak kalır

İlk bakışta Yol 2 "daha güvenli" göründü (botun trafiğine dokunmuyoruz). Ama derinlemesine düşününce, önemli bir itiraz ortaya çıktı: **Yol 2, gerçek bir bot-vari deseni (bir sayfa açılır açılmaz, insan tepki süresinden çok daha hızlı bir tıklama) veride bırakmak anlamına geliyordu — sadece bunu "görmezden gelmeyi" seçiyorduk, kaldırmıyorduk.** Bu, makine öğrenmesinde bilinen bir tuzakla aynı aileden: **bir metriği geçmek ile gerçekliği doğru temsil etmek aynı şey değildir.**

Sonunda Yol 1'i seçtik — ama "uygun bir sayı bulup ayarlamak" şeklinde değil. Kodun zaten her yerde kullandığı evrensel kuralı ("bir navigasyon → bir bekleme") sadece gözden kaçan bir yere **tutarlı şekilde** uyguladık. Yani yeni bir istatistik icat etmedik, var olan mantığı doğru yere yaydık.

Buradan öğrenilen ders — bu belki haftanın en önemli dersi

Bir modelin/sistemin bir testi geçmesi, o sistemin gerçekliği doğru yansıttığı anlamına gelmez. "Testi geçirmek için ayarlama" ile "gerçek bir kusuru, var olan tutarlı mantıkla düzeltmek" arasındaki fark, ileride yapay zeka ve güvenlik alanında sürekli karşına çıkacak bir ayrım. Sentetik veri/model davranışına, sadece bir testi rahatlatmak için doğrulanmamış yapı eklemek, o yapının kendisi yeni bir tespit edilebilir iz haline gelebilir.

9. İstatistiksel Doğrulama Sonuçları (KS Testi)

KS testi (Kolmogorov-Smirnov testi) nedir? İki veri kümesinin (örneğin botun ürettiği bekleme süreleri ile CICIDS'teki gerçek bekleme süreleri) aynı dağılımdan gelip gelmediğini istatistiksel olarak test eden bir yöntem. "p-değeri" 0.05'in üzerindeyse, "bu ikisi istatistiksel olarak birbirinden ayırt edilemez" denir (iyi haber); altındaysa "anlamlı bir fark var" denir.

9.1 Düzeltme öncesi ve sonrası karşılaştırma

Persona	Düzeltme Öncesi (KS p-değeri)	Düzeltme Sonrası (KS p-değeri)	Sonuç
Gezinen (browsing)	0.0000	0.4442	✓ Düzeldi
Arayan (searching)	0.0039	0.2470	✓ Düzeldi
Form Dolduran (formfilling)	0.1355 (zaten iyiydi)	0.1355 (değişmedi)	✓ Etkilenmedi (beklenen)

9.2 Yol boyunca çıkan ekstra bir hata — "kirli veri" dersi

Sorun

Düzelتمeyi test etmek için yeni bir capture (trafik kaydı) yaptığımızda, sonuç beklediğimiz kadar iyi çıkmadı. Araştırdıkça, kullandığımız analiz script'inin, klasördeki **TÜM** eski ve yeni kayıt dosyalarını (tarih ayrımı yapmadan) topladığını bulduk — yani düzeltme öncesi ve sonrası veriler **karişmiş**, yanıltıcı bir sonuç üretmişti.

Çözüm

Zaman damgalarına bakarak eski dosyaları tespit edip sildik, sadece yeni (düzeltme sonrası) veriyle tekrar analiz ettik — bu sefer tabloda gördüğün temiz sonuçlar çıktı.

Buradan öğrenilen ders

Bir "önce/sonra" karşılaştırması yaparken, karşılaştırdığın iki verinin gerçekten **sadece test ettiğin değişikende** farklı olduğundan emin olmak gerekir — aksi halde yanlış bir sonuca (ya "düzeldi" ya da yanlışlıkla "az düzeldi") ulaşabilirsin.

10. RITA Beacon Testleri — Kapsamlı Araştırma

Bu, haftanın en uzun ve en çok "neden böyle çıktı" sorusu sorduğumuz bölümü. Amaç basitti:

botun trafiği ile gerçek bir insanın trafiği, RITA'nın gözünde farklı mı?

Ama cevabı bulmak, birçok yanlış patikayı elemeyi gerektirdi.

10.1 İlk şüpheli sonuç

`MixedTrafficRunner`'ın ürettiği trafiği izole bir şekilde test ettiğimizde, beacon skoru **0.638** çıktı (eşik 0.50'nin üstünde, "Low" seviyesinde tespit). Bu, sabahki genel ölçümden (0.275) belirgin şekilde yüksekti — "bugünkü kod değişiklikleri botu daha 'bot-varı' mı yaptı?" sorusu doğdu.

10.2 Kontrol grubu — kendi trafiğimi ölçtüm

Bunu tek başına yorumlamak yanlış olurdu (karşılaştırma noktası yoktu) — bu yüzden aynı yöntemle

kendi gerçek gezinmemi

(elle, tarayıcıdan) izole olarak ölçtük. Sonuç: **0.272** — bot'tan düşük. Bu, ilk kez "belki gerçekten bot daha düzenli" hipotezini destekledi.

10.3 Ama adil bir karşılaştırma değildi — örneklem büyüklüğü sorunu

Dikkatlice baktığımızda, bot testinde

230 bağlantı

, insan testinde sadece **7 bağlantı** vardı (çünkü gerçek tarayıcımın ön belleği açtı, tekrar ziyaretlerde ağa hiç istek gitmedi). Bu adaletsizliği gidermek için, botun 230 bağlantısından **rastgele sadece 7'sini** seçip aynı büyüklükte bir karşılaştırma yaptık.

Bulgu

Botun rastgele 7-bağlantılı alt kümesi de **0.615** çıktı — insanın 7 bağlantısından (0.272) hâlâ belirgin şekilde yüksek. Bu, "örneklem büyüklüğü farkı" hipotezini çürüttü — sorun sayı farkı değildi.

10.4 Eşzamanlılık hipotezi de test edildi

`MixedTrafficRunner` aynı anda 3 bot çalıştırıyordu — belki bu karışıklık RITA'yı şaşırtıyordu? Botu **tek, sıralı** (eşzamanlılık olmadan) çalıştırıp yeniden ölçtük: **0.568** — hâlâ yüksek. Bu hipotez de çürüdü.

10.5 Yavaş gezinen insan testi — beklenmedik bir sonuç

Son bir kontrol için, kendi gezinmemi

bilerek çok yavaş

(sayfalar arası 15-40 saniye bekleyerek) tekrarladık. Sonuç: **0.892** — botun HİÇBİR versiyonundan daha yüksek, "Medium" seviyesinde!

10.6 Genel tablo ve nihai değerlendirme

Test	Beacon Skoru	Seviye
Genel ölçüm (sabah)	0.275	None
Bot, 3-eşzamanlı	0.638	Low
Bot, tek-sıralı	0.568	Low
Bot, 7-bağlantı örnekleme	0.615	Low
İnsan, hızlı gezinme	0.272	None
İnsan, yavaş gezinme	0.892	Medium

Nihai Sonuç

Skorlar 0.27 ile 0.89 arasında dağınık çıktı, hiçbirisi kesin bir "High" (0.90+) seviyesine net şekilde girmedi, ve bir insan testi bot testlerinin çoğundan daha yüksek çıktı. Bu, net ve tutarlı bir "bot her zaman insan'dan daha şüpheli görünür" ilişkisinin **olmadığını** gösteriyor. RITA, bu trafik türüne karşı ne kesin bir "zararsız" ne kesin bir "şüpheli" diyebiliyor — kararsız kalıyor.

Bu bölümün en önemli dersi

Bu sonuç, ilk bakışta "hayal kırıklığı" (kesin bir cevap bulamadık) gibi görünebilir, ama aslında **projenin ana tezini destekleyen çok değerli bir bulgu**: klasik, istatistiksel/kural-tabanlı bir güvenlik aracı (RITA), bu tür trafiği **kesin biçimde ayırt edemiyor** — ne zaman "evet bu bir bot" ne zaman "hayır, insan" diyeceğini bilemiyor. Bu tam olarak, Faz 2'de eğiteceğin Autoencoder'ın

doldurması gereken boşluk: klasik araçların kararsız kaldığı yerde, öğrenilmiş bir istatistiksel model daha ince bir ayırım yapabilir mi?

10.7 Yol boyunca çözülen teknik engeller (kısaca)

- **RITA komut sözdizimi:**

Eski dokümantasyondaki komutlar (`rita show-beacons` gibi) yeni RITA sürümünde (v5.1.2) çalışmıyordu — yeni sürüm hepsini tek bir `rita view` komutunda birleştirmiş. Doğru sözdizimini `--help` çıktısından bulduk.

- **Terminal TTY sorunları:**

`sudo` gerektiren bazı SSH komutları, parola sorabilmek için özel bir bayrak (`-t`) gerektiriyordu — bu birkaç kez unutulup hızlıca düzeltildi.

- **Zeek checksum uyarısı:**

Ağ kartının "checksum offloading" özelliği yüzünden, kaydedilen paketlerin bazı alanları (özellikle Pi'nin gönderdiği yanıtların boyutu) Zeek tarafında hep "0" görünüyordu. Bu, gerçek bir ağ sorunu değil, sadece paket-yakalama anındaki teknik bir görüntüleme kısıtlaması.

11. Genel Metodolojik Dersler

Bu hafta boyunca, tek tek hatalardan daha önemli olan, tekrar tekrar uyguladığımız bir **çalışma disiplini**

oluşturdu. Bunları maddeler halinde topluyorum, çünkü bunlar sadece bu projeye değil, ileride yapacağın her teknik/araştırma çalışmasına uygulanabilir:

1. **Önce ölç, sonra hüküm ver.**

"Bu bozuk görünüyor" dediğimiz her an, hemen düzeltmeye geçmek yerine, önce kanıt topladık (log'lar, zaman damgaları, izole testler).

2. **Kontrol grubu olmadan sonuç çıkarma.**

Bot trafiğinin beacon skorunu tek başına yorumlamak yanlış olurdu — insan trafiğiyle karşılaştırmak, asıl anlamı ortaya çıkardı.

3. **Değişkenleri tek tek izole et.**

RITA testinde eşzamanlılığı, örneklem büyüklüğünü, gezinme hızını birer birer değiştirip test ettik — hepsini aynı anda değiştirseydik, hangisinin etkili olduğunu asla bilemezdik.

4. **"Testi geçirmek" ile "gerçeği düzeltmek" arasındaki farkı bilerek seç.**

PRODUCT_DETAIL tartışmasında olduğu gibi, bazen daha "kolay" görünen çözüm (sadece ölçümü ayarlamak) gerçek sorunu gizler.

5. **Bir aracın çıktısına körü körüne güvenme.**

`timedatectl` 'in yanlış "senkron değil" uyarısı gibi — bazen aracın kendisi eksik/yanlış bilgi verebilir, konuya özel doğru yöntemle çapraz kontrol gerekir.

6. **Veri kirliliğine karşı her zaman şüpheli ol.**

Eski ve yeni dosyaların karışması, gerçek bir düzelmeyi "az düzeldi" gibi gösterebilir — her analiz öncesi veri kaynağının temiz olduğunu doğrulamak gerekir.

7. **Belirsiz/kararsız bir sonuç, başarısızlık değildir.**

RITA testinin "kesin cevap yok" sonucu, aslında projenin en güçlü bulgularından biri oldu.

12. Sonuç ve Sıradaki Adımlar

Bu hafta, botun altyapısı daha sağlam (kalıcı Pi ayarları), davranışı daha gerçekçi (üç kod hatası düzeltildi, istatistiksel doğrulama güçlendi), ve klasik güvenlik araçlarına karşı davranışı çok daha iyi anlaşılmış (RITA'nın kararsızlığı belgelendi) bir noktaya geldi.

Henüz yapılmayanlar (not edildi, öncelik değil şimdilik):

- PRODUCT_DETAIL düzeltmesinin, resmi/tez-seviyesinde daha büyük bir örneklemle (N=10) tekrar doğrulanması.
- RITA'nın conn.log süre ölçümündeki tutarsızlığın (checksum offloading ile ilişkili olabilir) ayrıca araştırılması.
- Faz 2 — Zeek loglarından özellik çıkarımı (feature extraction) ve Autoencoder eğitimi**
 - projenin bir sonraki büyük adımı, henüz başlanmadı.

Bu rapor, 5 Temmuz 2026 tarihli çalışma oturumunun kapsamlı bir özetidir. Detaylı teknik loglar ve ham veriler context.md ve IDS-Analysis/ klasöründe saklanmaktadır.